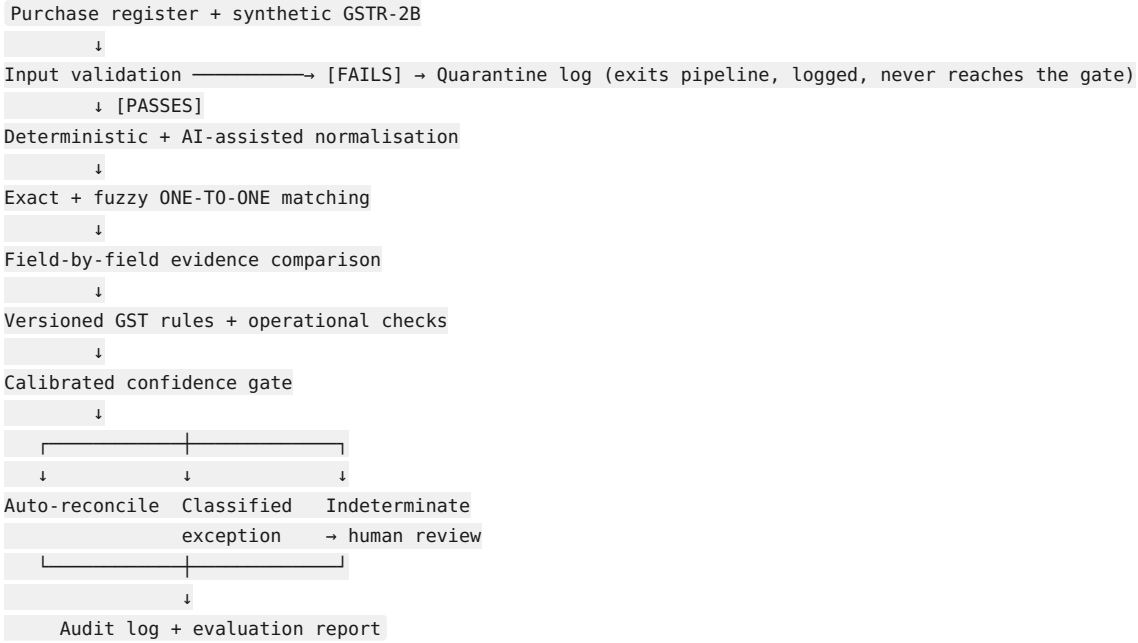
Exception Ledger — Final System Architecture & Pipeline (Locked, v2)

Status: This supersedes `Architecture.md` v1. This is the version to build against. Every stage below reflects the refinements made in review — one-to-one matching, split deterministic/AI normalisation, evidence comparison as its own stage, rules vs. operational checks reported separately, and a quarantine path for invalid input.

---

1. Pipeline overview



Two exit paths from the pipeline, not one: **quarantine** (bad input, never scored) and the **three-way gate outcome** (valid input, scored). Keeping these separate matters — a quarantined record is a data-quality problem, not a reconciliation problem, and folding it into "indeterminate" would misrepresent what the exception report is actually saying.

---

2. Stage-by-stage detail

2.1 Input validation

Module: `src/validation.py`

Checks, in order, before anything else touches the record:

- Required fields present (invoice_id, vendor_gstin, invoice_date, taxable_value, at least one of cgst/sgst/igst)
- GSTIN format valid (15 characters, correct checksum pattern, valid state-code prefix)
- Date parseable and within a plausible range
- Numeric fields are actually numeric and non-negative

On failure: write to `quarantine_log` (see Section 3.3) with the specific validation error, and the record goes no further. It is not counted in the match rate, the exception count, or the indeterminate count — it gets its own reported number: "N records quarantined, M% of batch."

2.2 Deterministic + AI-assisted normalisation

Module: `src/normalization.py`

Split explicitly into two functions, not one:

- `normalize_deterministic(record)` — pure code: GSTIN checksum correction/standard casing, date format standardization, numeric rounding. No LLM call.
- `normalize_ai_assisted(record)` — Claude API call, JSON-only system prompt, used only for messy free text (vendor name variants, OCR-style artifacts). Returns a cleaned string; **never returns a match decision or a confidence score.**

2.3 Exact + fuzzy one-to-one matching

Module: `src/matcher.py`

This is the stage that most needed fixing. Algorithm:

1. For every purchase-register record, compute a candidate score against every GSTR-2B record: exact fields (invoice number, GSTIN, amount within ₹1) score highest; fuzzy fields (vendor-name similarity via `rapidfuzz`, date within a tolerance window) score lower but non-zero.
2. Build the full candidate score matrix.
3. **Greedy one-to-one assignment:** sort all candidate pairs by score descending, assign the highest-scoring pair first, then remove both records from further consideration, repeat. This guarantees no GSTR-2B record is claimed by more than one purchase-register record and vice versa.
4. Any purchase-register record left unassigned after this process is explicitly marked `no_candidate_found` — this is a valid, expected output, not an error, and it feeds directly into the rule engine as an "absence" case (Section 2.5).

A simple greedy assignment is intentional over a full optimal bipartite solver (Hungarian algorithm) — it's easier to explain and verify in a review, and at this batch size the difference in outcome is negligible.

2.4 Field-by-field evidence comparison

Module: `src/evidence.py`

Takes a matched pair (or a `no_candidate_found` singleton) and produces a plain diff object — **no interpretation, no verdict:**

```
        Evidence(
            invoice_id="INV-1042",
            fields={
                "amount": {"pr_value": 11800.00, "2b_value": 10800.00, "delta": 1000.00, "match": False},
                "gstin":  {"pr_value": "27ABCDE...", "2b_value": "07ABCDE...", "match": False},
                "date":   {"pr_value": "2026-03-14", "2b_value": "2026-03-14", "match": True},
                "invoice_number": {"match": True},
            },
            candidate_found=True
        )
```

This object is what gets logged verbatim in the audit log's matched/mismatched fields — the rule engine consumes it but does not modify it.

2.5 Versioned GST rules + operational checks

Module: `src/rule_engine.py` + `src/rules/rules_v2026_04.yaml`

Two distinct rule categories, reported separately in the final output — do not merge them:

Classification rules (what happened):

- GSTIN header mismatch (state-code prefix differs → likely CGST/SGST vs. IGST filing error)
- Credit-note netting (amount delta matches a known credit-note pattern)
- `no_candidate_found` → classified as late-filed-supplier or invoice-removed-post-claim, disambiguated by whether the invoice appears in a prior-period synthetic snapshot

Operational checks (what to do about it, and by when):

- Rule 88D: is this mismatch inside or outside the 7-day response window (using a `simulated_current_date` field in the dataset)?
- DRC-01C threshold: does the cumulative ITC variance for this vendor exceed the auto-notice trigger?

Both categories are versioned in the same YAML file, but the evaluation report shows them as two separate tables, not

one blended list.

2.6 Calibrated confidence gate

Module: `src/confidence.py` + `src/gate.py`

Confidence score is built from the evidence object directly — count of matching fields, weighted by field importance (GSTIN and amount matter more than date formatting) — **not an LLM's self-reported confidence number.**

Calibration protocol (unchanged from v1, restated for clarity): 1. Dataset split 70% calibration / 30% frozen test, before any tuning. 2. On the calibration split only, sweep threshold values and pick the one that best separates true matches from true mismatches (using the `ground_truth.csv` labels, which the pipeline itself never sees during normal operation — only the evaluation script sees them). 3. The frozen 30% is touched exactly once, at the end, to produce the reported numbers. No re-tuning after that run.

Three outcomes:

Outcome	Condition
Auto-reconcile	Confidence ≥ threshold, evidence shows a clean match
Classified exception	Confidence below threshold, but the rule engine confidently assigns a named category
Indeterminate → human review	Rule engine cannot confidently assign any category

2.7 Audit log + evaluation report

Module: `src/audit_log.py` + `src/report.py`

Audit log: one row per record that passed validation — `record_id, evidence_snapshot, rule_id_fired, confidence_score, action, reviewer_decision (nullable), timestamp`.

Evaluation report (frozen test set only):

- Overall match rate (exact / fuzzy / rule-classified, broken out separately)
- Classification-rule exceptions, listed by named category
- Operational-check flags (Rule 88D window status, DRC-01C threshold status), listed separately from the classification exceptions
- Indeterminate count
- Quarantined-record count, reported as its own line, not folded into any of the above

---

3. Data structures

3.1 Core CSVs (unchanged from v1)

`purchase_register.csv`, `gstr2b.csv`, `ground_truth.csv` — schemas as previously defined, with one addition: both source CSVs now carry a `simulated_current_date` and enough date fields to compute the Rule 88D window and DRC-01C threshold checks.

3.2 Evidence object

As shown in Section 2.4 — this is the object type passed between matching, evidence comparison, and the rule engine. Define it once as a dataclass or TypedDict; every downstream stage consumes the same shape.

3.3 Quarantine log

`quarantine_log` (separate SQLite table from the audit log): `record_id, validation_error, raw_record_snapshot, timestamp`. Reported as a distinct count, never merged into the exception or indeterminate numbers.

---

4. Repo structure

```
exception-ledger/
  data/
```

```
    generate_dataset.py
    purchase_register.csv
    gstr2b.csv
    ground_truth.csv
  src/
    validation.py
    normalization.py       # normalize_deterministic() + normalize_ai_assisted()
    matcher.py              # one-to-one greedy assignment
    evidence.py
    rule_engine.py
    rules/rules_v2026_04.yaml
    confidence.py
    gate.py
    audit_log.py
    quarantine_log.py
    report.py
    pipeline.py             # orchestrates all stages, end to end
  tests/
    test_matcher.py         # verify one-to-one property explicitly
    test_rule_engine.py
    test_validation.py
  README.md
```

---

5. Build order (start here, in this sequence)

1. `data/generate_dataset.py` — the defect-plan generator (Section 3 of the earlier defect-distribution spec). Nothing else can be verified without this existing first.
2. `src/validation.py` + `src/quarantine_log.py` — small, self-contained, easy first win.
3. `src/normalization.py` — deterministic half first (no API dependency), AI-assisted half second.
4. `src/matcher.py` — build the one-to-one assignment and **write `test_matcher.py` alongside it immediately**, asserting no record appears in more than one pair. This is the stage most likely to silently misbehave if not tested directly.
5. `src/evidence.py` — straightforward once matching produces pairs.
6. `src/rule_engine.py` + `rules_v2026_04.yaml` — encode classification rules first, operational checks second.
7. `src/confidence.py` + `src/gate.py` — implement the calibration protocol exactly as specified; do not skip the calibration/frozen-test split even during early development.
8. `src/audit_log.py` + `src/report.py` — wire everything into `pipeline.py` as the final orchestration step.

Do not build the report formatting or presentation layer before step 8 is producing real numbers — there is nothing to present yet, and time spent there earlier is time taken from the matcher and rule engine, which are the components the entire evaluation depends on.